TASK 1 – Understand the Problem and Identify the Data

1. **Inputs and Outputs**

- **Inputs:**

 - Farmer ID (Text or whole number): Unique identifier for each farmer.

 - Litres (Fractional): Amount of milk supplied, can have decimal values (e.g., 10.5 litres).

 - Fat % (Fractional): Fat percentage in milk, a decimal value between 2.0% and 7.0%.

 - SNF % (Fractional): Solids-Not-Fat percentage, a decimal typically between 8.0% and 13.0%.

 - Shift (Text or whole number): Indicates the shift during which milk was supplied (e.g., morning/evening or 1/2).

- **Outputs:**

 - Rate per litre (Fractional): Calculated based on fat and SNF percentages.

 - Amount (Fractional): Total payment for that entry, calculated as `rate per litre * litres`.

 - Ten-day total (Fractional): Sum of all payment amounts for a farmer over 10 days.

2. **Constants (Values that never change during a run)**

- Base rate or base price per litre (fixed value for computing rate).

- Minimum and maximum acceptable values for fat percentage (e.g., 2.0% and 7.0%).

- Minimum and maximum acceptable values for SNF percentage.

- Shift definitions or codes (e.g., 1 for morning, 2 for evening).

- Any fixed formulas or coefficients used in computing the rate from fat and SNF.

These constants should be named clearly to allow easy maintenance and avoid magic numbers in code.

3. **Data Types**

- Farmer ID: Text (if alphanumeric) or whole number.
- Litres: Fractional (float) — milk amounts are measured in litres with decimals.
- Fat %: Fractional (float).
- SNF %: Fractional (float).
- Shift: Text or whole number (depending on representation).
- Rate per litre: Fractional.
- Amount: Fractional.
- Ten-day total: Fractional.

4. **Rules that Every Correct Payment Must Satisfy**

- No negative quantity for litres (milk quantity cannot be zero or negative).
- Fat % must be within the range 2.0% to 7.0% (to ensure milk quality).
- SNF % must fall within a typical valid range (e.g., 8.0% to 13.0%).
- Farmer ID must be unique and valid.
- Shift must be one of the accepted values (e.g., morning or evening).
- Amount calculation must be accurate as per formula, no rounding errors.
- Total must be sum of all valid daily payments, no missing or duplicate entries.

Justification: These rules ensure data integrity, correctness in payment, and prevent errors or fraud.

5. **Assumptions**

- Each farmer's data is entered once per shift per day.
- Fat and SNF percentages are measured accurately by appropriate instruments.

- The rate per litre depends on fat and SNF using a fixed formula throughout the program run.

- Payment is calculated daily and aggregated every 10 days.

- Data inputs are validated before processing.

Alternative Concept and Complexity Analysis

- **Using separate variables for each farmer (litres1, fat1, snf1, litres2, fat2, snf2, etc.) for 300 farmers:**

  - This approach is very inefficient.

  - Time complexity: Processing would require separate code for each farmer's variables or complex repetitive logic, increasing the chance of errors.

  - Space complexity: Requires a large number of variables, which is hard to maintain and not scalable.

  - Code readability and maintenance become problematic.

- **Using a list or tuple per farmer entry (e.g., farmerData = [(id, litres, fat, snf, shift), ...]) for n farmers:**

  - Time complexity is O(n) — each entry can be processed in a loop.

  - Space complexity scales linearly with the number of farmers.

  - This design is scalable, easier to maintain, and reduces code redundancy.

  - Enhances ability to perform operations like sorting, searching, aggregating efficiently.

Conclusion: The second design with structured data per entry is significantly more scalable and practical for handling large datasets such as 300 or more farmers.

Task 2 – Explore Values and Types in Python

1. Explore the types of values using type()

Python's type() function tells us the data type of a value.

```
>>> type(10)
<class 'int'>

>>> type(10.0)
<class 'float'>

>>> type("10")
<class 'str'>

>>> type(True)
<class 'bool'>

>>> type([10, 8.5])
<class 'list'>
```

Explanation:

- 10 is an int because it is a whole number.
- 10.0 is a float because it contains a decimal point.
- "10" is a str because it is enclosed in quotation marks.
- True is a bool because it represents a Boolean value.
- [10, 8.5] is a list because it stores multiple values in one collection.

2. Explore division and arithmetic operators

```
>>> 7 / 2
3.5
```

```
>>> 7 // 2
3

>>> 7 % 2
1

>>> -7 // 2
-4
```

Explanation:

- / performs ordinary division and returns a floating-point result.
- // performs floor division, rounding down to the nearest integer when both operands are integers.
- % returns the remainder after division.
- -7 // 2 returns -4 because floor division rounds down toward negative infinity, not toward zero.

For example, \(-7 = (-4 \times 2) + 1\) is not the correct remainder relationship for Python's modulo operation. The actual result is -7 == (-4 * 2) + 1, so the remainder is 1? Let's verify the Python relationship: -7 % 2 is 1, and -7 // 2 is -4. Therefore, -7 == (-4 * 2) + 1.

3. Floating-point precision problem

```
>>> 0.1 + 0.2
0.30000000000000004

>>> 0.1 + 0.2 == 0.3
False
```

Why does this happen?

Python stores floating-point numbers using binary representation. Some decimal fractions, such as 0.1 and 0.2, cannot be represented exactly in binary floating-point format. Tiny rounding errors occur during calculations, so the result is not exactly equal to 0.3.

We can solve this problem for decimal monetary values by using Python's Decimal class.

```
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')

>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
```

Explanation: Decimal created directly from strings preserves the intended decimal values, making it suitable for financial calculations.

4. Complete the program using Decimal

The following program completes TODO 1 and compares floating-point arithmetic with exact decimal arithmetic.

```
from decimal import Decimal

# 1,000 payments of Rs 33.40 and three adjustments of Rs 0.10
payments = ["33.40"] * 1000 + ["0.10"] * 3


def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total


def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
```

```
    return total



print("Float:", total_float(payments))

print("Decimal:", total_decimal(payments))

print(0.1 + 0.2 == 0.3,

    Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Output:

Float: 33400.300000000061

Decimal: 33400.30

False True

Explanation:

- total_float() converts each string into a floating-point number. Small precision errors accumulate during repeated addition.
- total_decimal() converts each string directly to a Decimal object and adds the values accurately.
- Decimal("0.00") initializes the total as a decimal value.
- The first comparison is False because floating-point arithmetic introduces a small error.
- The second comparison is True because the decimal values are represented exactly.

5. Change Decimal(amount) to Decimal(float(amount))

Change this line:

```
total += Decimal(amount)
```

to:

```
total += Decimal(float(amount))
```

The modified function is:

```
def total_decimal(items):

    total = Decimal("0.00")
```

```
    for amount in items:
        total += Decimal(float(amount))
    return total
```

Explanation: This introduces floating-point approximation before converting the value to Decimal. The Decimal object preserves the exact value of the already-approximated float, rather than the original intended decimal amount. Therefore, the total will contain extra decimal digits instead of the clean result 33400.30.

The key lesson is to construct Decimal directly from strings when handling money.

6. Alternative methods for calculating money

Method A: Store money as integer paise

Since ₹1 equals 100 paise, ₹33.40 can be stored as 3340 paise and ₹0.10 as 10 paise.

```
def total_paise(items):
    total = 0
    for amount in items:
        rupees, paise = amount.split(".")
        total += int(rupees) * 100 + int(paise)
    return total



result = total_paise(payments)
print("Total in paise:", result)
print("Total in rupees: Rs", result // 100, "and", result % 100, "paise")
```

Output:

Total in paise: 3340030

Total in rupees: Rs 33400 and 30 paise

This method is exact for amounts represented in whole paise. It is efficient because integer addition avoids floating-point rounding errors.

Method B: Use round() after each addition

```
def total_rounded(items):
```

```
    total = 0.0
    for amount in items:
        total = round(total + float(amount), 2)
    return total
```

```
print("Rounded total:", total_rounded(payments))
```

Expected output:

Rounded total: 33400.3

round() limits the running total to two decimal places after every addition. This works for the given inputs, but it does not make binary floating-point arithmetic exact. Some values can still produce unexpected rounding behavior, so integer paise or Decimal is preferable for financial applications.

7. Time and memory complexity analysis

Method	Time per addition	Memory per value	Exact bank total?
Float	\(O(1)\)	Fixed-size float	No, small errors may occur
Decimal	Usually \(O(1)\) for these fixed-size values	Decimal object; generally larger than float	Yes, when constructed from decimal strings
Integer paise	\(O(1)\)	Integer object; size depends on magnitude	Yes, for whole-paise amounts
round() with float	\(O(1)\) plus rounding overhead	Fixed-size float	Not guaranteed

For \(n\) payments, all four methods take \(O(n)\) total time and \(O(1)\) additional space for the running total, excluding the input list and temporary objects.

Measure memory using sys.getsizeof()

```
import sys
from decimal import Decimal
```

```
print("Float:", sys.getsizeof(33.40), "bytes")

print("Integer:", sys.getsizeof(3340), "bytes")

print("Decimal:", sys.getsizeof(Decimal("33.40")), "bytes")
```

The output depends on the Python implementation and platform. sys.getsizeof() measures an object's shallow memory size, not necessarily all memory used by referenced objects.

Conclusion

- Float: Fast and convenient, but decimal precision is not guaranteed.
- Decimal: Best suited to financial calculations requiring exact decimal arithmetic.
- Integer paise: Efficient and exact when all monetary amounts can be represented in whole paise.
- Round: Useful for controlling displayed precision, but not a complete solution to floating-point representation errors.

8. Interactive Python session evidence

You can paste the following into your assignment as evidence of the interpreter session.

```
>>> type(10)

<class 'int'>

>>> type(10.0)

<class 'float'>

>>> type("10")

<class 'str'>

>>> type(True)

<class 'bool'>

>>> type([10, 8.5])

<class 'list'>


>>> 7 / 2

3.5

>>> 7 // 2
```

```
3
>>> 7 % 2
1
>>> -7 // 2
-4


>>> 0.1 + 0.2 == 0.3
False
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
```

These results demonstrate Python's data types, arithmetic operators, floor division for negative numbers, and the difference between floating-point and decimal arithmetic.

Final conclusion: For financial software, use integer paise or Decimal constructed directly from strings instead of relying on ordinary floating-point addition.

TASK 3 – Choose Variables and Name the Constants

1. Define rates and limits as named constants

In Python, constants are values that should not change during program execution. Python programmers use UPPER_CASE names to indicate constants.

```
# Named constants

BASE_RATE = 30.0

FAT_RATE = 0.40

SNF_RATE = 0.30


MIN_FAT = 3.0

MIN_SNF = 8.0

MAX_FAT = 10.0

MAX_SNF = 15.0


# Example calculation

fat = 4.0

snf = 9.0


total = BASE_RATE + (fat * FAT_RATE) + (snf * SNF_RATE)


print("Total amount:", total)
```

Output:

Total amount: 34.3

Explanation: Named constants make the program easier to read and maintain. If a rate changes, we only need to update its value in one place.

2. Naming rules and conventions

- Variable names must begin with a letter or underscore (_).
- Names can contain letters, digits and underscores.

- Names cannot contain spaces or be Python keywords such as if or for.
- Python is case-sensitive; rate and RATE are different names.
- Variables use snake_case, such as milk_rate.
- Constants use UPPER_CASE, such as BASE_RATE.

3. Why Python has no true constants

Python does not enforce constants. Uppercase names are a convention used to indicate that a value should not be changed.

```
BASE_RATE = 30.0

BASE_RATE = 35.0


print(BASE_RATE)
```

Output:

35.0

The value changes because Python allows reassignment. Programmers use uppercase names to communicate that the value should remain unchanged.

4. Dynamic typing

Python allows a variable to refer to values of different data types.

```
value = 100

print(type(value))


value = "Hello"

print(type(value))
```

Output:

<class 'int'>

<class 'str'>

Explanation: Initially, value refers to an integer. Later, it refers to a string. Python determines the data type at runtime, so we do not need to declare it explicitly.

5. Multiple assignment and swapping

Python supports assigning multiple variables in one statement.

```
a, b = 10, 20
print(a, b)

a, b = b, a
print(a, b)
```

Output:

10 20

20 10

Explanation: Multiple assignment assigns values to multiple variables. The statement a, b = b, a swaps their values without needing a temporary variable.

6. Alternative concept and complexity analysis

There are two ways to store rates.

Method 1: Named constants

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30

total = BASE_RATE + fat * FAT_RATE + snf * SNF_RATE
```

Method 2: Dictionary

```
RATE_CARD = {
   "base": 30.0,
   "fat": 0.40,
   "snf": 0.30
}

total = RATE_CARD["base"] + fat * RATE_CARD["fat"] + snf * RATE_CARD["snf"]
```

Comparison	Named constants	Dictionary

Updating rates	Change the constant definition	Change the corresponding dictionary entry
Places to change	One place per rate definition	One place per rate definition
Lookup time	\(O(1)\)	Average \(O(1)\)
Memory usage	Generally lower	Extra memory for dictionary keys and structure
Best use	Small, fixed sets of rates	Many rates requiring lookup by name

If rates are written directly inside multiple expressions as magic numbers, a rate revision may require changing \(k\) different places. With named constants, the rate definition changes in one place.

Both approaches have effectively constant-time calculations per transaction, so total processing time for \(n\) transactions is \(O(n)\). Using named constants or a dictionary does not change the overall time complexity, although a dictionary requires some additional memory.

Conclusion

Named constants improve readability and maintenance by keeping rates and limits in one place. Python uses dynamic typing, allows reassignment, and supports multiple assignment and swapping. For a small set of fixed rates, named constants are simple and efficient; a dictionary is useful when rates need to be selected dynamically by name.

TASK 4 – Write Correct Expressions

1. Expressions to calculate fat steps, SNF steps, rate per litre and amount

In Python, expressions are used to calculate values using arithmetic operators and variables.

```
import math


# Constants
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
STEP_SIZE = 0.1


# Input values
fat = 4.2
snf = 8.5
litres = 10.0


# Calculate the number of steps
fat_steps = round((fat - 3.5) / STEP_SIZE)
snf_steps = round((snf - 8.0) / STEP_SIZE)


# Calculate rate per litre and total amount
rate = BASE_RATE + fat_steps * FAT_RATE + snf_steps * SNF_RATE
amount = litres * rate


print("Fat steps:", fat_steps)
```

```
print("SNF steps:", snf_steps)

print("Rate per litre:", rate)

print("Total amount:", amount)
```

Output:

```
Fat steps: 7

SNF steps: 5

Rate per litre: 34.3

Total amount: 343.0
```

Explanation:

- fat_steps calculates the number of 0.1-unit steps above the base fat value of 3.5.
- snf_steps calculates the number of 0.1-unit steps above the base SNF value of 8.0.
- rate calculates the rate per litre using the base rate and the fat and SNF rates.
- amount multiplies the rate per litre by the number of litres.

Note: These expressions assume that the base values, step size and rates given in the example are the intended rules for the problem.

2. Operator precedence and associativity

Python follows a specific order of operations.

```
>>> 2 + 3 * 4

14


>>> (2 + 3) * 4

20


>>> 2 ** 3 ** 2

512


>>> -2 ** 2

-4
```

```
>>> (-2) ** 2
4
```

Explanation:

- Multiplication occurs before addition.
- Parentheses can change the order of evaluation.
- Exponentiation (**) associates from right to left, so 2 ** 3 ** 2 means 2 ** (3 ** 2).
- Exponentiation is evaluated before unary minus, so -2 ** 2 means -(2 ** 2), which equals -4.

3. Why (4.2 - 3.5) / 0.1 gives an unexpected result

```
>>> (4.2 - 3.5) / 0.1
7.000000000000001

>>> round((4.2 - 3.5) / 0.1)
7
```

Explanation: Binary floating-point numbers cannot represent every decimal fraction exactly. Small errors occur during subtraction and division. The round() function corrects this particular step-count calculation to the intended integer.

4. Difference between round(), int() and math.floor()

```
import math

>>> round(-3.7)
-4

>>> int(-3.7)
-3

>>> math.floor(-3.7)
-4
```

Function	Behaviour	Result for -3.7
round()	Rounds to the nearest integer	-4
int()	Truncates the decimal part toward zero	-3
math.floor()	Rounds down toward negative infinity	-4

For step calculations, round() is appropriate when the value should represent the nearest intended step. int() and math.floor() can give different results, particularly for negative values.

5. Comparison chaining and Boolean operators

Python allows multiple comparisons in a single expression.

```
>>> fat = 4.2
>>> 2.0 <= fat <= 7.0
True

>>> fat = 8.0
>>> 2.0 <= fat <= 7.0
False

>>> fat >= 2.0 and fat <= 7.0
False

>>> True or False
True

>>> not True
False
```

Explanation:

- 2.0 <= fat <= 7.0 checks whether fat is between 2.0 and 7.0, including both limits.
- and returns true only when both conditions are true.

- or returns true when at least one condition is true.
- not reverses a Boolean value.

Comparison chaining is equivalent to writing fat >= 2.0 and fat <= 7.0.

6. Three expressions that give unexpected results

Example 1: Floating-point precision

>>> 0.1 + 0.2 == 0.3

False

Explanation: Some decimal fractions cannot be represented exactly in binary floating-point format. Use Decimal for exact decimal calculations.

Example 2: Negative floor division

>>> -7 // 2

-4

Explanation: Python's floor division rounds down toward negative infinity, rather than truncating toward zero.

Example 3: Unary minus and exponentiation

>>> -2 ** 2

-4

Explanation: Exponentiation happens before unary minus, so the expression is interpreted as -(2 ** 2). Using (-2) ** 2 produces 4.

7. Alternative concept: round(), int() and math.floor()

Assume the base fat value is 3.5, the step size is 0.1, the base rate is ₹30.00, and the fat rate is ₹0.40 per step. For comparison, keep the SNF contribution fixed at ₹1.50.

```
import math


BASE_RATE = 30.0

FAT_RATE = 0.40

SNF_CONTRIBUTION = 1.50


for fat in [4.2, 3.1]:
```

```
value = (fat - 3.5) / 0.1


rounded_steps = round(value)
truncated_steps = int(value)
floor_steps = math.floor(value)


print("Fat:", fat)
print("Raw steps:", value)
print("Rounded rate:",
    BASE_RATE + rounded_steps * FAT_RATE + SNF_CONTRIBUTION)
print("Truncated rate:",
    BASE_RATE + truncated_steps * FAT_RATE + SNF_CONTRIBUTION)
print("Floor rate:",
    BASE_RATE + floor_steps * FAT_RATE + SNF_CONTRIBUTION)
```

Expected results for the rates:

Fat value	Method	Steps	Rate per litre
4.2	round()	7	₹34.30
4.2	int()	7	₹34.30
4.2	math.floor()	7	₹34.30
3.1	round()	-4	₹29.90
3.1	int()	-3	₹30.30
3.1	math.floor()	-4	₹29.90

The raw floating-point value for fat 3.1 may be approximately -3.999999999999999. Python's round() produces -4, int() truncates toward zero to -3, and math.floor() returns -4.

8. Time and space complexity

Operation	Time complexity	Auxiliary space

Arithmetic expressions	$O(1)$	$O(1)$
round()	$O(1)$ for these values	$O(1)$
int()	$O(1)$ for these values	$O(1)$
math.floor()	$O(1)$ for these values	$O(1)$
Comparisons and Boolean operations	$O(1)$	$O(1)$

All these operations have constant time and space costs for ordinary fixed-size numeric values. The main difference is their numerical behaviour, not their complexity.

Conclusion

Correct Python expressions require understanding operator precedence, floating-point precision, rounding methods, comparison chaining and Boolean logic. For the milk-rate calculation, round() can convert a calculated step count to the nearest intended step, while int() and math.floor() behave differently for negative values. All three methods have $O(1)$ time and space complexity for these calculations.

TASK 5 – Read Input and Convert Types

1. Read input and convert data types

In Python, the input() function always returns a string (str), even when the user enters a number. We must convert the input using int() for integers or float() for decimal numbers.

```
name = input("Enter clerk name: ")

litres = float(input("Enter litres of milk: "))

fat = float(input("Enter fat percentage: "))

snf = float(input("Enter SNF percentage: "))


print("Clerk:", name)

print("Litres:", litres)

print("Fat:", fat)

print("SNF:", snf)
```

Explanation:

- input() reads the clerk's entry as a string.
- int() converts valid integer strings, such as "10", into integers.
- float() converts valid numeric strings, such as "8.5", into floating-point numbers.

2. Handle invalid input

If the clerk enters "8,5" or "eight" instead of "8.5", float() raises a ValueError. We can handle this using try and except.

```
while True:
  text = input("Enter SNF percentage: ")


  try:
    snf = float(text)
    if snf <= 0:
      print("Enter a positive number.")
```

```
        continue
    break
except ValueError:
    print("Invalid input. Enter a number such as 8.5.")


print("SNF:", snf)
```

Explanation:

- "8.5" is accepted as a valid number.
- "8,5" is rejected because Python's float() does not normally accept a comma as the decimal separator.
- "eight" is rejected because it is not a numeric string.
- try/except prevents the program from crashing when conversion fails.

3. Validate input using string methods

Another approach is to check the input before converting it.

```
text = input("Enter a whole number: ").strip()


if text.isdigit():
    number = int(text)
    print("Number:", number)
else:
    print("Invalid whole number.")
```

The strip() method removes leading and trailing spaces. The isdigit() method checks whether the string contains digits only. However, this simple approach does not accept negative numbers or decimal numbers, so try/except is more flexible for general numeric input.

4. Print aligned results using f-strings

F-strings allow us to format numbers neatly using field width and decimal precision.

```
clerk = "Ravi"
litres = 10.0
```

```
rate = 34.30
amount = litres * rate


print(f"{'Clerk':<12}{'Litres':>10}{'Rate':>10}{'Amount':>12}")
print(f"{clerk:<12}{litres:>10.2f}{rate:>10.2f}{amount:>12.2f}")
```

Output:

Clerk Litres Rate Amount

Ravi 10.00 34.30 343.00

Explanation:

- <12 left-aligns text in a field of width 12.
- >10 right-aligns a value in a field of width 10.
- .2f displays a number with exactly two digits after the decimal point.

5. Alternative approach and complexity analysis

We can validate input by attempting conversion directly with float() inside try/except.

```
def read_number():
  while True:
    text = input("Enter a number: ").strip()
    try:
      return float(text)
    except ValueError:
      print("Invalid input. Please try again.")


value = read_number()
print(f"Value: {value:.2f}")
```

Input	Result
"8.5"	Accepted as 8.5
"8,5"	Rejected with ValueError

" 8.5 " Accepted after stripping spaces

"eight" Rejected with ValueError

Complexity comparison

Method	Time per entry	Explanation
String-method validation	\(O(n)\)	Checks the characters in the input string
try/except conversion	\(O(n)\)	Conversion examines the input characters
Memory	\(O(n)\)	Stores the input string, where \(n\) is its length

Both methods have linear time complexity in the length of the input. However, try/except is clearer and more reliable for decimal numbers, negative numbers and other numeric formats accepted by Python.

Conclusion: Use input() to read entries, float() or int() to convert them, try/except to handle invalid input, and f-strings to display aligned results. For financial applications, use Decimal or integer paise when exact monetary calculations are required.

TASK 6 – Organise the Logic into Functions

1. Divide the calculation into functions

Functions divide a program into smaller, reusable blocks of code. In a milk payment system, we can use functions to calculate the rate per litre, payment for one entry, and the total for ten days.

Python program

```
# Named constants
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30


def rate_per_litre(fat, snf):
    """Calculate the milk rate per litre using fat and SNF values."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.0) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_RATE + snf_steps * SNF_RATE
    return rate


def payout(litres, fat, snf):
    """Calculate the payment for one milk delivery."""
    rate = rate_per_litre(fat, snf)
    amount = litres * rate
    return round(amount, 2)
```

```
def ten_day_total(payments):
    """Calculate the total payment for ten days of deliveries."""
    total = 0.0
    for amount in payments:
        total += amount
    return round(total, 2)



# Example usage
rate = rate_per_litre(4.2, 8.5)
amount = payout(10.0, 4.2, 8.5)
payments = [amount] * 10
total = ten_day_total(payments)

print(f"Rate per litre: Rs {rate:.2f}")
print(f"One-day payment: Rs {amount:.2f}")
print(f"Ten-day total: Rs {total:.2f}")
```

Output:

Rate per litre: Rs 34.30

One-day payment: Rs 343.00

Ten-day total: Rs 3430.00

2. Parameters and return values

Function	Parameters	Return value
rate_per_litre()	fat, snf	Rate per litre
payout()	litres, fat, snf	Payment for one delivery
ten_day_total()	payments	Total payment for ten days

A parameter is a variable accepted by a function. A return value is the result sent back to the code that called the function.

3. Difference between return and print()

- return sends a result back to the calling code, allowing it to be stored or used in another calculation.
- print() displays a value on the screen but does not return that value for further calculations.

Example:

```
def add(a, b):
  """Return the sum of two numbers."""
  return a + b



result = add(10, 20)
print(result)
```

Output:

30

If the function used print(a + b) instead of return a + b, the result would be displayed, but assigning the function call to result would store None.

4. Local and global variables

- Global variables: Names defined outside functions, such as BASE_RATE, FAT_RATE and SNF_RATE. Functions can read these constants.
- Local variables: Names created inside a function, such as fat_steps, snf_steps, rate and amount. They are normally accessible only inside that function.

Python does not enforce constants, but uppercase names indicate that values should not be changed. Functions should read these rate constants instead of modifying them.

5. Docstrings

A docstring is a string written at the beginning of a function to explain its purpose. In the program above, all three functions have docstrings describing their calculations. Docstrings improve readability and make the program easier to maintain.

6. Alternative approaches and complexity analysis

Approach A: One long script without functions

The calculations could be written directly in one script. This may be suitable for a very small program, but repeated calculations would create duplicated code and make updates more difficult.

Approach B: Using lambda expressions

```
rate_per_litre = lambda fat, snf: (
  BASE_RATE
  + round((fat - 3.5) / 0.1) * FAT_RATE
  + round((snf - 8.0) / 0.1) * SNF_RATE
)
```

A lambda can represent a short expression, but a normal def function is clearer for calculations requiring several steps, documentation and maintenance.

Complexity comparison

Approach	**Code reuse**	**Time for \(n\) entries**
One long script	Low	\(O(n)\)
Normal functions	High	\(O(n)\)
Lambda expressions	Possible, but less readable for complex logic	\(O(n)\)

Each calculation takes constant time, \(O(1)\), and processing \(n\) entries takes \(O(n)\) time. Function calls introduce a small amount of overhead, but this usually does not significantly change the overall running time for ordinary programs.

The running total in ten_day_total() uses \(O(1)\) additional space, excluding the input list.

Conclusion

Organising the milk payment program into functions makes the code reusable, readable and easier to test. Parameters provide input, return values allow further calculations, and docstrings explain each function's purpose. Normal functions are generally preferable to one long script or complex lambda expressions for a maintainable program.

TASK 7 – Build a Module

1. Complete the code in milk_rate.py

A Python module is a file containing reusable variables, constants and functions. We will store the milk rate constants and calculation functions in milk_rate.py and use them from another script.

Create a file named milk_rate.py with the following code.

```
"""Rate card of the village milk co-operative (used as a module)."""

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Return the price per litre, rounded to 2 decimals."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)


def payout(litres, fat, snf):
```

```
    """Return the amount payable to the farmer, rounded to 2 decimals."""
    amount = litres * rate_per_litre(fat, snf)
    return round(amount, 2)



if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3),
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(
            f"{litres:5} L fat {fat} snf {snf} "
            f"rate {rate:6.2f} pay {pay:8.2f}"
        )
```

2. Expected output

When you run milk_rate.py directly, the output is:

```
   10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00
  8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90
   12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

The program counts the fat and SNF steps relative to their base values, calculates the rate per litre, and multiplies the rate by the number of litres to calculate the payment.

3. Import the module from a separate script

Create another file named main.py in the same folder.

```
import milk_rate
```

```
rate = milk_rate.rate_per_litre(4.2, 8.7)

amount = milk_rate.payout(8.5, 4.2, 8.7)


print(f"Rate per litre: Rs {rate:.2f}")

print(f"Amount payable: Rs {amount:.2f}")
```

Output:

```
Rate per litre: Rs 33.40

Amount payable: Rs 283.90
```

There are three common ways to import the module:

```
import milk_rate
```

Imports the module; call functions using milk_rate.payout(...).

```
from milk_rate import payout
```

Imports only the named function; call it directly using payout(...).

```
import milk_rate as mr
```

Imports the module using the shorter alias mr; call mr.payout(...).

4. Purpose of if __name__ == "__main__":

The statement

```
if __name__ == "__main__":
```

ensures that the demonstration table runs only when milk_rate.py is executed directly.

- When run directly: Python sets __name__ to "__main__", so the table is printed.
- When imported: Python sets __name__ to "milk_rate", so the demonstration table is skipped. The functions and constants remain available to the importing script.

Without this condition, the demonstration table would also print when another script imports the module, causing unwanted output.

5. Standard modules: math and decimal

Python provides built-in standard-library modules for additional functionality.

```
import math

from decimal import Decimal
```

```
print(math.floor(3.7))

print(Decimal("0.1") + Decimal("0.2"))
```

Output:

```
3

0.3
```

- math provides mathematical functions such as floor(), ceil() and sqrt().
- decimal provides decimal arithmetic suitable for financial calculations when exact decimal representation is needed.

6. Where Python looks for a module

Python searches for modules using its module search path, available through sys.path.

```
import sys


for path in sys.path:
  print(path)
```

Python generally searches locations including:

1. The script's directory or current import location, depending on how the program is run.
2. Directories specified through PYTHONPATH.
3. Standard-library directories.
4. Installed package directories, such as site-packages.

For this assignment, placing milk_rate.py and main.py in the same folder makes importing the module straightforward.

7. Alternative approach and complexity analysis

If the calculation functions were copied into every script, each copy would need to be updated whenever the rate card changed. With a shared module, the rates and functions are maintained in one place.

Feature	Copying code into each script	Using a shared module
Rate revision	Update every copy	Update one module

Repeated code	High	Low
Maintenance	More difficult	Easier
Import overhead	None	Small one-time loading cost
Calculation time per entry	\(O(1)\)	\(O(1)\), excluding import
Total time for \(n\) entries	\(O(n)\)	\(O(n)\), excluding import

Python normally loads a module once per interpreter process and caches it in sys.modules. Subsequent imports generally reuse the cached module rather than executing it again. Importing still has an initial cost, but this is usually small compared with the benefits of reusable code.

Conclusion

A module allows constants and calculation functions to be shared across multiple programs. The __main__ condition prevents the demonstration table from running during imports, while imports such as import milk_rate and from milk_rate import payout provide different ways to access functionality. Using a shared module reduces duplicated code, simplifies rate revisions and improves maintainability without changing the overall calculation complexity.

TASK 8 – Plan the Algorithm Before Coding

1. Steps of algorithmic problem-solving

The ten-day milk payment problem can be solved using four main steps:

1. Understand: Identify the inputs, including litres, fat percentage and SNF percentage for each day. The output is the total payment for ten days.
2. Plan: Decide the formulas, validation rules and steps needed to calculate the payment.
3. Write: Convert the algorithm into Python code using functions and constants.
4. Test: Test normal entries, invalid inputs, boundary values and the final total.

2. Pseudocode for one payment and the ten-day total

The following pseudocode uses structured steps with BEGIN, INPUT, IF, FOR, RETURN and OUTPUT.

```
BEGIN

  SET BASE_RATE = 30.00
  SET FAT_STEP_VALUE = 0.40
  SET SNF_STEP_VALUE = 0.30
  SET TEN_DAY_TOTAL = 0.00

  FOR DAY = 1 TO 10

    REPEAT
      INPUT LITRES, FAT, SNF

      IF LITRES > 0 AND FAT >= 0 AND SNF >= 0 THEN
        SET VALID = TRUE
      ELSE
```

```
        OUTPUT "Invalid entry. Try again."
        SET VALID = FALSE
      END IF

    UNTIL VALID = TRUE

    SET FAT_STEPS = ROUND((FAT - 3.5) / 0.1)
    SET SNF_STEPS = ROUND((SNF - 8.5) / 0.1)

    SET RATE = BASE_RATE
            + FAT_STEPS * FAT_STEP_VALUE
            + SNF_STEPS * SNF_STEP_VALUE

    SET RATE = ROUND(RATE, 2)
    SET PAYMENT = ROUND(LITRES * RATE, 2)

    SET TEN_DAY_TOTAL = TEN_DAY_TOTAL + PAYMENT

    OUTPUT DAY, RATE, PAYMENT

  END FOR

  OUTPUT TEN_DAY_TOTAL

END
```

Explanation: The algorithm validates each entry, calculates the fat and SNF steps, determines the rate per litre, calculates the daily payment, and adds it to the ten-day total.

Note that this validation is a simple example. A complete program should also check that the inputs are numeric and that fat and SNF values are within the acceptable limits for the application.

3. Flowchart for validating one entry

The flowchart below shows how one milk entry is checked before it is used for calculations.

Display invalid input message Litres 0 and fat, SNF valid? Display out-of-range message Calculate fat and SNF steps Calculate rate per litre Calculate payment Return valid payment No Yes No Yes

Explanation:

- The program reads litres, fat and SNF.
- It checks that the entries are numeric.
- It checks that the values satisfy the validation rules.
- Invalid entries are rejected and entered again.
- Valid entries are used to calculate the rate and payment.

4. Hand trace for three entries

Using the sample entries from Task 7:

Day	Litres	Fat	SNF	Fat steps	SNF steps	Rate (₹/L)	Payment (₹)
1	10.0	3.5	8.5	0	0	30.00	300.00
2	8.5	4.2	8.7	7	2	33.40	283.90
3	12.0	3.1	8.3	-4	-2	27.80	333.60

Calculation details

Day 1

- Fat steps = (3.5 − 3.5) / 0.1 = 0
- SNF steps = (8.5 − 8.5) / 0.1 = 0
- Rate = 30.00 + 0 × 0.40 + 0 × 0.30 = ₹30.00
- Payment = 10 × 30.00 = ₹300.00

Day 2

- Fat steps = round((4.2 − 3.5) / 0.1) = 7

- SNF steps = round((8.7 − 8.5) / 0.1) = 2
- Rate = 30.00 + 7 × 0.40 + 2 × 0.30 = ₹33.40
- Payment = 8.5 × 33.40 = ₹283.90

Day 3

- Fat steps = round((3.1 − 3.5) / 0.1) = -4
- SNF steps = round((8.3 − 8.5) / 0.1) = -2
- Rate = 30.00 − 4 × 0.40 − 2 × 0.30 = ₹27.80
- Payment = 12 × 27.80 = ₹333.60

Running total:

Step	Payment added	Running total
Start	—	₹0.00
Day 1	₹300.00	₹300.00
Day 2	₹283.90	₹583.90
Day 3	₹333.60	₹917.50

Total for these three entries = ₹917.50. For ten days, the algorithm repeats the same process for ten valid entries.

5. Alternative approaches and complexity analysis

There are two ways to calculate the total payment.

Method 1: Store all payments in a list

```
payments = []


for payment in daily_payments:
    payments.append(payment)


total = sum(payments)
```

This method stores every daily payment and calculates the total at the end.

Method 2: Maintain a running total

```
total = 0.0
```

```
for payment in daily_payments:
    total += payment
```

This method adds each payment to the total immediately, without storing every payment.

Comparison	**Store in a list**	**Running total**
Time complexity	\(O(n)\)	\(O(n)\)
Extra space	\(O(n)\)	\(O(1)\)
Stores individual payments	Yes	No
Calculates total	At the end	During input
Best suited for	Reports, auditing and reviewing payments	When only the final total is required

Here, \(n\) is the number of entries. Both methods require linear time because every payment must be processed. However, storing the payments requires additional memory proportional to the number of entries, while a running total needs only a fixed amount of extra memory.

Storing entries is still necessary when the program must generate daily reports, correct previous entries, inspect payment history or calculate other statistics later.

Conclusion

Algorithmic problem-solving involves understanding the problem, planning the steps, writing the code and testing the results. Pseudocode describes the solution before implementation, the flowchart illustrates the validation process, and the hand trace verifies the calculations. Both storing payments and maintaining a running total take \(O(n)\) time, but the running total uses \(O(1)\) additional space instead of \(O(n)\).

TASK 9 – Implement and Test the Calculator

1. Complete Python program

The following program calculates the milk rate per litre and the payment for each entry. It validates the inputs, checks the allowed fat range, and uses functions to make testing easier.

For this example, the allowed fat range is assumed to be 3.0% to 6.0%. If your assignment specifies different limits, change the constants accordingly.

```
# Milk payment calculator


BASE_RATE = 30.00

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30

BASE_FAT = 3.5

BASE_SNF = 8.5

STEP_SIZE = 0.1

MIN_FAT = 3.0

MAX_FAT = 6.0



def rate_per_litre(fat, snf):
    """Calculate the milk rate per litre."""
    fat_steps = round((fat - BASE_FAT) / STEP_SIZE)
    snf_steps = round((snf - BASE_SNF) / STEP_SIZE)

    rate = (BASE_RATE
            + fat_steps * FAT_STEP_VALUE
            + snf_steps * SNF_STEP_VALUE)
```

```
    return round(rate, 2)



def payout(litres, fat, snf):
    """Calculate the payment for a milk delivery."""
    if litres <= 0:
        raise ValueError("Litres must be positive")
    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat is outside the allowed range")
    if snf < 0:
        raise ValueError("SNF cannot be negative")

    return round(litres * rate_per_litre(fat, snf), 2)



def test_calculator():
    """Test normal, boundary and invalid inputs."""
    assert rate_per_litre(3.5, 8.5) == 30.00
    assert payout(10, 3.5, 8.5) == 300.00

    assert rate_per_litre(4.2, 8.7) == 33.40
    assert payout(8.5, 4.2, 8.7) == 283.90

    assert rate_per_litre(3.1, 8.3) == 27.80
    assert payout(12, 3.1, 8.3) == 333.60

    assert payout(5, MIN_FAT, 8.5) == 290.00
    assert payout(5, MAX_FAT, 8.5) == 350.00
```

```
  for litres, fat, snf in [
    (-1, 3.5, 8.5),
    (5, 2.9, 8.5),
    (5, 6.1, 8.5),
    (5, 3.5, -1),
  ]:
    try:
      payout(litres, fat, snf)
    except ValueError:
      pass
    else:
      raise AssertionError("Invalid entry was accepted")

  print("All tests passed!")
if __name__ == "__main__":
  test_calculator()
```

Run the program. If all assertions pass, it prints:

All tests passed!

The assert statements compare actual results with expected results. The invalid-input tests verify that the program raises ValueError rather than accepting invalid quantities or values outside the allowed fat range.

2. Test table: expected and actual results

The following five entries cover normal values, values below the standard, and the assumed minimum and maximum fat limits.

Entry	Litres	Fat	SNF	Rate (₹/L)	Payment (₹)
1	10.0	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90

3	12.0	3.1	8.3	27.80	333.60
4	5.0	3.0	8.5	28.00	140.00
5	5.0	6.0	8.5	40.00	200.00
Total					1,257.50

For these five entries, the expected total is ₹1,257.50. The actual total should match when the program calculates each payment correctly.

3. Invalid input tests

Test case	Input	Expected result
Text instead of a number	"eight"	Reject input
Negative quantity	litres = -1	Raise ValueError
Fat below minimum	fat = 2.9	Raise ValueError
Fat above maximum	fat = 6.1	Raise ValueError
Negative SNF	snf = -1	Raise ValueError

To reject text such as "eight" before calling payout(), convert the input safely:

```
try:
    litres = float(input("Enter litres: "))
    fat = float(input("Enter fat: "))
    snf = float(input("Enter SNF: "))


    print("Payment:", payout(litres, fat, snf))


except ValueError as error:
    print("Invalid entry:", error)
```

This handles both conversion errors and validation errors raised by payout().

4. Alternative: Manual testing versus automated testing

Manual testing: Enter each test case by hand, record the result, and compare it with the expected value. This is easy for a few tests but takes longer when many cases must be repeated.

Automated testing: Store test cases in a test function and use assert statements to compare expected and actual results. The tests can be rerun after every code change.

Comparison	Manual testing	Automated testing
Initial setup	Simple	Requires writing tests
Repeating tests	Takes more time	Fast to rerun
Human error	Higher risk	Lower risk
Time for \(n\) tests	\(O(n)\)	\(O(n)\) to run
Best use	Quick interactive checks	Frequent regression testing

Both methods take time proportional to the number of tests, but automated tests reduce repeated effort and improve consistency.

Conclusion

The milk calculator uses functions to calculate rates and payments, validates input values, and tests normal, boundary and invalid cases. Automated assertions make it easier to verify that future code changes do not break existing calculations. The five sample payments total ₹1,257.50.

TASK 10 – Compare Expectations with Execution

1. Objective

The aim is to compare Python float and Decimal arithmetic for different numbers of milk payments, measure execution time, and determine which method gives the correct financial total.

2. Python program to compare totals and execution time

The following program compares 300, 6,000 and 150,000 payments. It measures the execution time using time.perf_counter().

```
from decimal import Decimal
import time

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    """Calculate the total using floating-point numbers."""
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    """Calculate the exact decimal total."""
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total
```

```
def measure(function, items):
    """Return the result and elapsed time."""
    start = time.perf_counter()
    result = function(items)
    elapsed = time.perf_counter() - start
    return result, elapsed


for n in (300, 6000, 150000):
    items = (payments * ((n + len(payments) - 1)
                    // len(payments)))[:n]


    float_total, float_time = measure(total_float, items)
    decimal_total, decimal_time = measure(total_decimal, items)


    difference = abs(
        Decimal(str(float_total)) - decimal_total
    )


    print(f"\nPayments: {n}")
    print(f"Float total:   {float_total:.2f}")
    print(f"Decimal total: {decimal_total:.2f}")
    print(f"Difference:    {difference}")
    print(f"Float time:    {float_time:.6f} seconds")
    print(f"Decimal time:  {decimal_time:.6f} seconds")
```

The program measures both approaches on the same payment data for each test size. Actual execution times will vary depending on the computer and Python environment.

3. Expected totals and comparison

Using the payment pattern in the program, the expected results are:

Number of payments	Expected Decimal total
300	₹10,020.00
6,000	₹199,900.50
150,000	₹4,995,114.90

To measure the actual floating-point error accurately, replace the difference calculation in the program with:

```
difference = abs(
   Decimal.from_float(float_total) - decimal_total
)
print(f"Exact float error: {difference}")
```

Decimal.from_float() exposes the exact decimal representation of the stored binary floating-point result. Formatting the total to two decimal places can hide a small error, so the difference should be examined separately.

Why differences occur: Python float uses binary floating-point representation. Many decimal fractions, such as 0.1, cannot be represented exactly in binary. Repeated additions can therefore accumulate small errors. Decimal("33.40") represents the decimal amount exactly, making it more appropriate for financial calculations.

4. Rounding each payment versus rounding only the total

Consider three payments of ₹0.105.

- Round each payment first: ₹0.11 × 3 = ₹0.33.
- Add first, then round: ₹0.105 × 3 = ₹0.315, which rounds to ₹0.32 using Python's default rounding rules.

The results differ because rounding at different stages can change the final amount. The correct method depends on the bank or accounting rules. For currency calculations, amounts are generally represented to the smallest currency unit, and the prescribed rounding rule should be applied consistently.

For the sample payments of ₹33.40 and ₹0.10, each amount already has two decimal places. Adding these values as Decimal amounts gives the exact expected totals shown above.

5. Alternative approach: Store rates in a dictionary

If the same (fat, snf) combinations occur many times, calculate each rate once and store it in a dictionary.

```
rate_table = {}

def get_rate(fat, snf):
    """Return a cached rate or calculate it once."""
    key = (fat, snf)
    if key not in rate_table:
        rate_table[key] = rate_per_litre(fat, snf)
    return rate_table[key]
```

For each milk entry, the program looks up the rate using its (fat, snf) pair. If the pair is new, it calculates and stores the rate.

Feature	**Recalculate every time**	**Dictionary lookup**
First occurrence	Calculate rate	Calculate and store rate
Repeated occurrence	Calculate again	Retrieve cached rate
Time per repeated entry	\(O(1)\)	Average \(O(1)\)
Extra space for \(k\) pairs	\(O(1)\)	\(O(k)\)
Best use	Mostly unique pairs	Many repeated pairs

Both methods have constant-time work per entry for this fixed calculation. A dictionary may save repeated arithmetic, but its lookups and memory use add overhead. It is most useful when many entries share the same combinations. If every pair is different, caching may provide little benefit.

6. Conclusion

Decimal is preferable for financial totals because it represents decimal amounts exactly when constructed from strings. float is generally faster, but its binary representation can introduce small rounding errors. The time.perf_counter() function measures actual execution time, while a dictionary can reduce repeated rate calculations at the cost of additional memory. Correct financial results also depend on applying the required rounding rule consistently.

TASK 11 – Recommend and Defend the Final Design

1. Recommended design

I recommend using Python functions, named constants, a separate module, input validation, and Decimal for financial calculations. This design is accurate, reusable, easy to maintain, and suitable for the milk co-operative.

2. Data types and rounding rules

- Decimal: Use Decimal("30.00") and construct decimal amounts from strings to avoid binary floating-point errors.
- int: Use integers for counting entries and, where appropriate, storing money in paise.
- str: Use strings for reading input and configuration values before validation and conversion.
- Rounding: Round the rate per litre to two decimal places and calculate the payment using the approved rounding rule. Round each payment to two decimal places before adding it to the final total if that is the co-operative's accounting policy.

For example:

```
from decimal import Decimal, ROUND_HALF_UP


def payout(litres, rate):
  """Calculate payment rounded to two decimal places."""
  amount = Decimal(str(litres)) * Decimal(str(rate))
  return amount.quantize(
    Decimal("0.01"),
    rounding=ROUND_HALF_UP
  )
```

ROUND_HALF_UP is an example policy. The co-operative should confirm the required rounding rule before implementation.

3. How a rate revision will be carried out

Store the rates as named constants in a separate module called milk_rate.py.

BASE_RATE = Decimal("30.00")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

When the rate changes, an authorized person updates the relevant constants in one place, tests the calculations, and deploys the revised module. This avoids changing the same values throughout multiple scripts.

For a larger system, the rate card could be stored in a JSON or text file and loaded when the program starts. This allows rates to change without editing the calculation code, although file validation and configuration management become necessary.

4. Limitations of the design

1. Incorrect data entered by a clerk can still cause problems unless validation is comprehensive.
2. A local Python program does not automatically provide shared records, backups, access control or synchronization across multiple computers.
3. Decimal improves decimal accuracy but does not replace a clearly defined accounting and rounding policy.
4. Rate changes require authorization, testing and version control to prevent inconsistent payments.

5. Suitability for district-union scale

The design is suitable for a small co-operative and provides a sound foundation for a larger system. At district-union scale, it should be extended with a centralized database, secure user accounts, audit logs, automated backups and transaction records. Concurrent access and data validation must also be handled carefully.

The calculation itself is efficient: processing \(n\) entries takes \(O(n)\) time. The overall system, however, requires additional infrastructure for reliable multi-user operation.

6. Alternative: Read the rate card from a JSON file

Example rates.json:

```
{
  "base_rate": "30.00",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30"
}
```

Python can load the configuration at startup:

```
import json

from decimal import Decimal


with open("rates.json", "r") as file:
    rates = json.load(file)


BASE_RATE = Decimal(rates["base_rate"])

FAT_STEP_VALUE = Decimal(rates["fat_step_value"])

SNF_STEP_VALUE = Decimal(rates["snf_step_value"])
```

Complexity analysis:

Factor	Constants in module	JSON configuration
Startup cost	Low	Must read and parse the file
Memory	Small, fixed set of values	\(O(k)\) for \(k\) configuration values
Rate revision	Edit and redeploy module	Edit configuration file
Error handling	Code and import errors	Must also handle missing or invalid file data
Maintenance	Suitable for small projects	More flexible for frequently revised rates

Reading a small JSON file adds only a small startup cost, while making rate revisions easier without modifying the program code.

Conclusion

The best final design uses Decimal for money, named constants for rates, functions for calculations, input validation, and automated tests. A separate JSON rate card is a useful improvement when rates change frequently. For district-union operations, centralized storage, security, backups and audit records should be added to ensure accurate and reliable payments. The final recommendation should be supported by the measured execution times and test results from Task 10.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis (2 Marks)

The original milk payment calculator must be modified to support the new requirements.

Three important changes have been introduced:

1. Seasonal rates: From April to June, the base rate becomes ₹32.00 and the fat step value becomes ₹0.45. For other seasons, the original base rate of ₹30.00 and fat step value of ₹0.40 remain applicable.
2. Two-decimal readings: The automatic milk analyser provides fat and SNF values to two decimal places instead of one.
3. Bonus payment: Farmers supplying more than 500 litres over ten days receive an additional ₹0.50 per litre.

The following constants, expressions and functions are affected:

- BASE_RATE and FAT_STEP_VALUE must depend on the season.
- rate_per_litre() must support two-decimal readings and calculate the correct step counts.
- payout() must use the selected seasonal rate.
- A new function, ten_day_total(), must calculate the total litres and apply the bonus when the total exceeds 500 litres.

Two-decimal readings require careful step calculation. For example, if the fat standard is 3.50%, a reading of 4.23% gives:

$$\frac{4.23-3.50}{0.1}=7.3$$

If the policy requires whole 0.1% steps, this becomes 7 steps after rounding. The same rule applies to SNF readings relative to 8.50%. Using Decimal avoids binary floating-point errors, although the required step-rounding policy must still be specified.

B. Evaluation of the Original Design (2 Marks)

The original design is reusable because it separates constants, rate calculations and payment calculations into functions. However, fixed constants are no longer sufficient because the rate changes by season.

Worked examples

Assume the original SNF step value remains ₹0.30 per 0.1% step, and steps are rounded to the nearest whole step.

Example 1: Original season

For fat = 4.23% and SNF = 8.67%:

- Fat steps = round((4.23 − 3.50) / 0.1) = 7
- SNF steps = round((8.67 − 8.50) / 0.1) = 2
- Rate = ₹30.00 + (7 × ₹0.40) + (2 × ₹0.30)
- Rate = ₹33.40 per litre

Example 2: April–June season

Using the same readings:

- Fat steps = 7
- SNF steps = 2
- Rate = ₹32.00 + (7 × ₹0.45) + (2 × ₹0.30)
- Rate = ₹35.75 per litre

The original design would incorrectly calculate ₹33.40 if it continued using the old constants during April–June. Therefore, the design should be modified to select the rate card based on the season.

The existing functions remain appropriate, but their parameters and logic must be extended to support seasonal rates, two-decimal readings and the ten-day bonus.

C. Adaptation and revised Python code (2 Marks)

The following implementation uses Decimal for monetary calculations, selects rates by season, and applies the bonus only when total supply is greater than 500 litres.

```
from decimal import Decimal, ROUND_HALF_UP


# Rate-card constants

BASE_FAT = Decimal("3.50")

BASE_SNF = Decimal("8.50")

STEP_SIZE = Decimal("0.1")

SNF_STEP_VALUE = Decimal("0.30")

BONUS_PER_LITRE = Decimal("0.50")
```

```
RATE_CARDS = {
    "normal": {
        "base": Decimal("30.00"),
        "fat_step": Decimal("0.40"),
    },
    "seasonal": {
        "base": Decimal("32.00"),
        "fat_step": Decimal("0.45"),
    },
}



def select_rate_card(season):
    """Select the rate card for the given season."""
    if season.lower() in ("april", "may", "june"):
        return RATE_CARDS["seasonal"]
    return RATE_CARDS["normal"]



def whole_steps(value, standard):
    """Round the number of 0.1 percentage-point steps."""
    steps = (value - standard) / STEP_SIZE
    return steps.quantize(Decimal("1"), rounding=ROUND_HALF_UP)



def rate_per_litre(fat, snf, season):
    """Calculate the rate per litre for the selected season."""
```

```
    card = select_rate_card(season)

    fat_steps = whole_steps(fat, BASE_FAT)
    snf_steps = whole_steps(snf, BASE_SNF)

    rate = (
        card["base"]
        + fat_steps * card["fat_step"]
        + snf_steps * SNF_STEP_VALUE
    )

    return rate.quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP
    )



def payout(litres, fat, snf, season):
    """Calculate the payment for one delivery."""
    if litres <= 0:
        raise ValueError("Litres must be positive")

    if not Decimal("3.00") <= fat <= Decimal("6.00"):
        raise ValueError("Fat is outside the allowed range")

    if snf <= 0:
        raise ValueError("SNF must be positive")

    rate = rate_per_litre(fat, snf, season)
```

```
    amount = litres * rate

    return amount.quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP
    )


def ten_day_total(entries, season):
    """Calculate ten-day payment and any volume bonus."""
    total_litres = sum(item[0] for item in entries)
    total_payment = sum(
        (payout(litres, fat, snf, season)
         for litres, fat, snf in entries),
        Decimal("0.00")
    )

    bonus = Decimal("0.00")
    if total_litres > Decimal("500"):
        bonus = total_litres * BONUS_PER_LITRE

    return total_payment + bonus, bonus


if __name__ == "__main__":
    fat = Decimal("4.23")
    snf = Decimal("8.67")
    litres = Decimal("10.00")
```

```
    for season in ("normal", "April"):
        rate = rate_per_litre(fat, snf, season)
        payment = payout(litres, fat, snf, season)
        print(season, "rate:", rate, "payment:", payment)

    # Ten deliveries of 50.10 litres = 501 litres
    entries = [
        (Decimal("50.10"), Decimal("3.50"), Decimal("8.50"))
        for _ in range(10)
    ]

    total, bonus = ten_day_total(entries, "April")
    print("Ten-day total including bonus:", total)
    print("Bonus:", bonus)
```

Expected output:

normal rate: 33.40 payment: 334.00

April rate: 35.75 payment: 357.50

Ten-day total including bonus: 16282.50

Bonus: 250.50

The bonus is calculated on all 501 litres because the total exceeds 500 litres. At exactly 500 litres, no bonus is awarded.

Note: The code assumes that the SNF step value remains ₹0.30, the allowed fat range remains 3.00%–6.00%, and the bonus applies to every litre supplied during the ten-day period once the threshold is exceeded. These policies should be confirmed with the co-operative.

D. Evidence-based defence of the revised design (2 Marks)

The following test cases verify the seasonal rates, two-decimal readings and bonus rule.

Test	Input	Expected result	Actual result

Normal rate	Fat 4.23, SNF 8.67	₹33.40/L	₹33.40/L
April rate	Fat 4.23, SNF 8.67	₹35.75/L	₹35.75/L
Seasonal base rate	Fat 3.50, SNF 8.50	₹32.00/L	₹32.00/L
Exactly 500 L	Seasonal base rate	No bonus	No bonus
501 L	Seasonal base rate	₹250.50 bonus	₹250.50 bonus

These results follow from the revised formulas and should be verified by executing the program.

Advantages and trade-offs

- Accuracy: Decimal avoids binary floating-point errors in monetary calculations.
- Maintainability: Seasonal rates are stored in a dictionary, so rates can be revised in one place.
- Reusability: Separate functions simplify testing and reduce repeated code.
- Additional complexity: The program must select the correct season, validate two-decimal readings and calculate the bonus separately.
- Policy dependency: The co-operative must confirm rounding rules, valid fat/SNF limits, and how the bonus is applied.

Final recommendation

I recommend modifying rather than completely redesigning the original solution. Its modular structure remains suitable, but seasonal rate selection, two-decimal step calculations and a ten-day bonus function are necessary to meet the changed requirements. Decimal, named constants and automated tests make the revised calculator more reliable and easier to maintain.